

Array type – Tip Niza

Sadrži metode za rad sa nizovima, koji su navedeni u nastavku:

`Array.AddItem(niz, vrednost) As Array`

Dodaje stavku u niz.

Parametri:

- *niz*: Ulazni niz. Ovaj argument se može izostaviti ako pozovete ovu metodu kao metodu proširenja promenljive niza.

- *vrednost*: Stavka koju želite da dodate posle poslednje stavke u nizu. Ako pošaljete niz, biće dodat kao jedna stavka u celini, tako da će novi indeks niza referisati na podniz. Pogledajte primere ispod.

Vraća: Novi niz sa novom dodatom stavkom. Ulazni niz se neće promeniti. Primeri:

```
a = {}  
a = Array.AddItem(a, "Prva stavka")  
a = Array.AddItem(a, "Druga stavka")  
a = Array.AddItem(a, "Treća stavka")
```

Nakon izvršavanja gornjeg koda, a će sadržati tri dodate stavke na indeksima 1, 2 i 3 respektivno.

Kao što primećujete, ponovo smo dodelili niz koji je vratila metoda `AddItem` promenljivoj a, inače se neće promeniti.

Imajte na umu da sVB može zaključiti da je tip a niz, jer je a inicijalizovan praznim nizom. Ovo vam omogućava da koristite metode niza kao metode proširenja promenljive a, stoga se gornji kôd može prepisati kao:

```
a = {}  
a = a.AddItem("Prva stavka")  
a = a.AddItem("Druga stavka")  
a = a.AddItem("Treća stavka")
```

Što je malo kraće, ali i dalje nije najbolje. Bolje je umesto toga koristiti sintaksu indeksa:

```
a[1] = "Prva stavka"  
a[2] = "Druga stavka"  
a[3] = "Treća stavka"
```

Ipak, možemo bolje koristiti inicijalizator niza:

```
a = {  
    "Prva stavka",  
    "Druga stavka",  
    "Treća stavka"  
}
```

Ali to ne znači da ponekad nećete koristiti `AddItem`. Korisno je kada želite da dodate stavku nizu bez poznavanja njegovog indeksa, tako da možete izbeći da radite ovo:

```
a[a.Count + 1] = "Nova stavka"
```

Gde smo dodali novu stavku na indeks niza pored njene poslednje. Zar ne biste radije koristili ovo umesto toga?:

```
a = a.AddItem("Nova stavka")
```

Takođe, ova metoda je poželjna kada želite da klonirate niz u drugi, plus dodavanje nove stavke novom nizu, kao što je:

```
b = a.AddItem("Nova vrednost")
```

Nakon izvršavanja gornje komande, `a` ostaje kakvo je bilo bez ikakvih promena, dok će `b` imati iste stavke `a` plus stavku "Nova vrednost" koja je novo dodata stavkama. To je kratka alternativa ovog koda:

```
b = a ' Kopiraj stavke iz `a` u `b`  
b[b.Count + 1] = "Nova vrednost"
```

Dakle, metoda `AddItem` može biti korisna na kraju krajeva. Imajte na umu da možete dodati niz kao stavku:

```
a = {}  
a = a.AddItem(  
{"Prva stavka", "Druga stavka", "Treća stavka"}  
)
```

Nakon izvršavanja gornjeg koda, `a` će sadržati samo jednu stavku na indeksu 1, što je dati niz. U stvari, gornji kôd je ekvivalentan:

```
a = {  
{"prva stavka", "Druga stavka", "Treća stavka"}  
}
```

Array.AddItem(niz, stavke) As Array

Dodaje više elemenata u niz.

Parametri:

- **niz**: Ulazni niz. Ovaj argument se može izostaviti ako pozovete ovu metodu kao metodu proširenja promenljive niza.

- **stavke**: Niz koji sadrži stavke za dodavanje svake od njih kao jedne stavke na kraj niza.

Vraća: Novi niz sa dodatim datim stavkama. Ulazni niz se neće menjati. Primeri:

```
a = {1, 2, 3}  
a = Array.AddItem(a, {4, 5})  
b = a.AddItem({100, 200, {300, 400}}, 500)
```

Nakon izvršavanja gornjeg koda, `a` će sadržati:

```
{1, 2, 3, 4, 5}
```

a `b` će sadržati:

```
{1, 2, 3, 4, 5, 100, 200, {300, 400}, 500}
```

Array.AddKeyValue(niz, ključ, vrednost) As Array

Dodaje stavku u niz, sa datim ključem i vrednošću

Parametri:

- **niz**: Ulazni niz. Ovaj argument se može izostaviti ako pozovete ovu metodu kao metodu proširenja promenljive niza.

- **ključ**: Ključ stavke. Ako već postoji stavka sa ovim ključem, njena vrednost će biti izmenjena.

- **vrednost**: Vrednost stavke. Ova vrednost može biti niz, koji će biti dodat kao jedna stavka.

Vraća: Novi niz sa dodatom stavkom. Ulazni niz se neće menjati. Primeri:

Ovaj kôd će dodati dve stavke u niz, prva ima ključ „Ime“ sa vrednošću „Adam“, a druga ima ključ „Ocene“ sa vrednostima niza koje sadrže 4 ocene:

```
a = {}  
a = Array.AddKeyValue(a, "Ime", "Adam")  
a = a.AddKeyValue("Ocene", {75, 60, 82, 99})
```

Lakše je i efikasnije prepisati gornji kôd kao:

```
a["Ime"] = "Adam"  
a["Ocene"] = {75, 60, 82, 99}
```

Takođe možete koristiti operator pretraživanja rečnika `!` da biste izrazili ključeve kao dinamička svojstva, uz podršku za automatsko dopunjavanje u uređivaču koda:

```
a!Ime = "Adam"  
a!Ocene = {75, 60, 82, 99}
```

Array.Append(niz, vrednost)

Dodaje stavku u niz. Slično je metodi `AddItem`, ali ovde će dati niz biti direktno promenjen, tako da je ova metoda brža kada želite da napravite veliki niz, ali budite oprezni jer će uticati na referentni niz iz kojeg se kopira trenutni niz! Pogledajte temu o performansama nizova za više informacija. Imajte na umu da sVB poziva ovu metodu da popuni niz kada koristite inicijalizator niza {}, tako da inicijalizator nije samo kraći već i brži od dodavanja pojedinačnih elemenata pomoću indeksa niza[]. Takođe, ako želite da izmenite stavku u velikom nizu, nemojte koristiti indeks jer će kopirati ceo niz. Postoji još jedna brza metoda referenciranja koja radi kao stavka `AddNext` da bi se ovo uradilo, a to je metoda `SetItemAt`.

Napomena: ako ova metoda ne radi, verovatno je to zato što ste joj prosledili tekstualnu reprezentaciju niza, a ne stvarni niz. Ovo se dešava veoma retko, jer sVB automatski konvertuje tekst u niz. Ali ako ste nekako naišli na tako retku situaciju, možete je lako popraviti korišćenjem metode `Text.ToArray` da biste dobili niz iz njegove tekstualne reprezentacije.

Parametri:

- *niz*: Ulazni niz. Ovaj argument se može izostaviti ako pozovete ovu metodu kao metodu proširenja promenljive niza.

- *vrednost*: Stavka koju želite da dodate posle poslednje stavke u nizu. Ako pošaljete niz, on će biti dodat kao jedna stavka u celini, tako da će novi indeks niza referisati na podniz. Primeri:

Možete videti ovu metodu u akciji u projektu *Array performance* – *Performanse niza* u folderu sa primerima. Koristi se u obrađivaču događaja `Button2_OnClick` za dodavanje 10.000 stavki u niz za manje od 20% sekunde.

Array.ContainsIndex(niz, indeks) As Boolean

Vraća vrednost da li niz sadrži navedeni indeks. Imajte na umu da je sVB niz zapravo rečnik gde je svaka stavka par ključ-vrednost. Ključ je jedinstveni string koji možete koristiti za pristup pojedinačnoj stavki u kolekciji, za postavljanje ili dobijanje njene vrednosti.

Dakle, za sVB, indeksi nizova su zapravo ključevi. To znači da se mogu koristiti brojevi ili stringovi kao indeksi, a numerički indeksirani brojevi ne moraju biti kontinuirani. Na primer, ovaj kôd je savršeno validan u sVB:

```
a[1] = 10  
a[7] = 9  
a["test"] = "OK"  
a[-1] = "Moguće je!"
```

Gornji kôd kreira niz koji sadrži indkse "1", "7", "test" i "-1". Činjenica da su svi indeksi zapravo string ključevi omogućava korišćenje negativnih indeksa poput `a[-1]` u gornjem kodu, što nije dozvoljeno za nizove u drugim jezicima!

U gornjem kodu nema indeksa između 1 i 7, i zato vam može biti potrebna metoda

ContainsIndex da biste proverili da li indeks (kao što je 4) postoji pre nego što ga upotrebite, jer ako pokušate da pročitate vrednost nepostojećeg indeksa, niz će vratiti prazan string "" i neće izazvati grešku. U većini situacija ovo možete smatrati dobrom stvari (i zapravo jeste), ali ponekad vam je potrebno da napravite razliku između stavke koja sadrži vrednost "" i nepostojeće stavke! Tada bi trebalo da koristite metodu ContainsIndex.

Parametri:

- *niz*: Niz koji treba proveriti. Ovaj argument se može izostaviti ako pozovete ovu metodu kao metodu proširenja promenljive niza.

- *indeks*: Broj ili string koji predstavlja indeks koji treba proveriti. Imajte na umu da su indeksi stringova neosetljivi na velika i mala slova, tako da ako niz ima indeks `test`, proveru za "test" i "Test" će vratiti True.

Vraća: True ako indeks postoji u nizu ili False u suprotnom. Primer:

```
a[1] = 10
a[3] = 20
If a.ContainsIndex(2) Then
    Forms.ShowMessage(a[2], "Primer")
Else
    Forms.ShowMessage("Stavka ne postoji", "Primer")
EndIf
(If Array.ContainsIndex(a, 4) = False Then
a[4] = 30
EndIf
```

Array.ContainsValue(niz, vrednost) As Boolean

Vraća da li niz sadrži navedenu vrednost.

Parametri:

- *niz*: Niz koji treba proveriti. Ovaj argument se može izostaviti ako pozovete ovu metodu kao metodu proširenja promenljive niza.

- *vrednost*: Vrednost koju treba proveriti.

Vraća: True ako je vrednost bila prisutna u navedenom nizu. False u suprotnom. Primeri:

```
x = Array.ContainsValue({1, 2, 3}, 2)
a = {1, 2, 3}
y = a.ContainsValue(4)
```

Nakon izvršavanja gornjeg koda, `x` će biti jednako True, a `y` će biti jednako False. Imajte na umu da su vrednosti stringova osetljivi na velika i mala slova, dok su ključevi stringova neosetljivi na velika i mala slova!

```
a["test"] = "Hi"
z = a.ContainsValue("Hi")
w = a.ContainsValue("hi")
```

Nakon izvršavanja gornjeg koda, `z` će biti jednako True, a `w` će biti jednako False.

Array.Count As Double

Ovo je svojstvo proširenja promenljivih niza. Dobija broj elemenata sačuvanih u nizu. Možete koristiti i metod Array.GetItemCount da biste obavili isti posao. *Vraća*: Broj elemenata navedenog niza. Primer:

```
a = {10, 20, 30}
n = a.Count ' n će sadržati 3
```

Array.EmptyArray As Array

Kreira prazan niz. Ovo je korisno kada treba da inicijalizujete promenljivu kao niz bez elemenata, što možete učiniti i sa praznim parom zagrada {}.

Inicijalizacija promenljive je važna da biste sVB-u rekli o njenom tipu, tako da možete koristiti metode ovog tipa kao metode proširenja ove promenljive.

Vraća: Prazan niz. Primer:

```
x = Array.EmptyArray  
count = x.Count
```

Takođe možete koristiti prazne zagrade {} da biste kreirali prazan niz:

```
x = {}  
count = x.Count
```

Nakon izvršavanja gornjeg koda, `count` će biti 0.

Array.Find(niz, vrednost, start, ignoreCase) As String

Pretražuje niz za datu vrednost.

Parametri:

- *niz*: Ulazni niz. Ovaj argument se može izostaviti ako pozovete ovu metodu kao metodu proširenja promenljive niza.

- *vrednost*: Stavka koju treba tražiti.

- *start*: Ceo broj koji predstavlja poziciju niza od koje treba početi pretragu. Ova vrednost mora biti > 0.

- *ignoreCase*: Kada je podešeno na True, pretraga će biti osetljiva na velika i mala slova.

Vraća: Indeks (ključ) stavke ako je pronađena, u suprotnom prazan string. Primer: Pretpostavimo da imamo ovaj niz:

```
a = {10, 20, 30, "Adam"}  
a["Ime"] = "Adam"  
a!Godine = 15
```

Hajde da izvršimo neke operacije pretrage nad njim. Rezultat pretrage ću napisati kao komentar pored svake linije. Ako je rezultat prazan string (stavka nije pronađena), komentar ću ostaviti prazan:

```
x = a.Find(30, 1, True)      ' 3  
x = a.Find(30, 4, True)      '  
x = a.Find("adam", 2, True)  ' 4  
x = a.Find("adam", 3, False) '  
x = a.Find("adam", 5, True)  ' Ime  
x = a.Find("10", 1, True)    '  
x = a.Find(15, 1, True)      ' Godine
```

Važnost drugog parametra (start) se pojavljuje kada tražimo „Adam“, jer niz sadrži dva parametra, jedan sa ključem „4“ i drugi sa ključem „Ime“. Ne možemo pronaći poslednji osim ako ne počnemo pretragu na 5. poziciji niza.

Značaj trećeg parametra (ignoreCase) se pojavljuje kada tražimo `adam`, dok niz sadrži samo Adam. Možemo izvršiti pretragu bez razlike između velikih i malih slova slanjem vrednosti True ovom parametru, u suprotnom će pretraga biti osetljiva na velika i mala slova i nećemo pronaći `adam`.

Vraćeni rezultat je ključ stavke. Stavka 30 postoji na trećoj poziciji niza i njen ključ je takođe 3, slično „Adam“ na 4. poziciji sa ključem 4, ali drugi „Adam“ postoji na poziciji 5 sa ključem

„Name“. Dakle, ključ stavke nije uvek njegova pozicija u nizu.

Array.GetAllIndices(niz) As Array

Dobija niz koji sadrži sve indekse za dati niz. Ovu metodu možete koristiti kao svojstvo proširenja promenljivih niza, sa nazivom Indeksi.

Parametar:

- *niz*: Niz čiji se indeksi traže.

Vraća: Niz popunjen svim indeksima (ključevima) navedenog niza. Vraćeni indeksi niza počinju od 1. Primeri: Pretpostavimo da imamo ovaj niz:

```
a = {10, 20, 30, "Adam"}  
a["Ime"] = "Adam"  
a!Godine = 15
```

Njegove indekse možemo dobiti ovako:

```
ids = a.Indices
```

Niz ids će biti:

```
{1, 2, 3, 4, "Ime", "Godine"}
```

I možemo dobiti id indekse niza ids samo da bismo bili sigurni:

```
ids2 = Array.GetAllIndices(a.Indices)
```

Niz ids2 će biti:

```
{1, 2, 3, 4, 5, 6}
```

Array.GetItemAt(niz, pozicija)

Dobija vrednost stavke koja postoji na datoj poziciji u nizu.

Parametri:

- *array*: Ulazni niz. Ovaj argument se može izostaviti ako pozovete ovu metodu kao metodu proširenja promenljive niza.

- *pozicija*: Broj koji predstavlja poziciju stavke u nizu, koji nije uvek isti kao njen indeks (ključ). Imajte na umu da pozicija mora biti > 0.

Vraća: Vrednost stavke (ako je pronađena), inače prazan string "". Primeri: Pretpostavimo da imamo ovaj niz:

```
a = {10, 20, 30, "Adam"}  
a["Ime"] = "Adam"  
a!Godine = 15
```

Možemo napisati for petlju za iteraciju kroz niz ovako:

```
For i = 1 To a.Count  
    TextWindow.Write(a.GetItemAt(i) + " ")  
Next
```

TextWindow će prikazati sledeći rezultat:

```
10 20 30 Adam Adam 15
```

Array.GetItemCount(niz) As Double

Dobija broj elemenata sačuvanih u nizu. Ovu metodu možete koristiti kao svojstvo promenljivih niza, sa imenom Count.

Parametar:

- *niz*: Niz za koji se traži broj.

Vraća: Broj elemenata navedenog niza. Primer:

```
a = {10, 20, 30}
n = Array.GetItemCount(a) ' n će sadržati 3
```

Array.GetKeyAt(*niz*, *pozicija*) As String

Dobija ključ elementa koji postoji na datoj poziciji niza.

Parametri:

- *niz*: Ulazni niz. Ovaj argument se može izostaviti ako pozovete ovu metodu kao metodu proširenja promenljive niza.

- *pozicija*: Broj koji predstavlja poziciju elementa u nizu, koji ne mora uvek biti isti kao njegov indeks (ključ). Imajte na umu da pozicija mora biti > 0.

Vraća: Ključ elementa ako je pronađen, inače prazan string. Primer: Pretpostavimo da imamo ovaj niz:

```
a = {10, 20, 30, "Adam"}
a["Ime"] = "Adam"
a!Starost = 15
```

Možemo napisati for petlju za iteraciju kroz niz ovako:

```
For i = 1 To a.Count
  TextWindow.Write(a.GetKeyAt(i) + " ")
Next
```

TextWindow će prikazati sledeći rezultat:

```
1 2 3 4 Ime Starost
```

Array.IndexOf(*niz*, *vrednost*, *početak*, *ignoreCase*) As Doble

Pretražuje dati niz za datu vrednost i vraća indeks te vrednosti u nizu.

Parametri:

- *niz*: Ulazni niz. Ovaj argument se može izostaviti ako pozovete ovu metodu kao metodu proširenja promenljive niza.

- *vrednost*: Stavka koju treba tražiti.

- *početak*: Ceo broj koji predstavlja indeks niza od kog treba početi pretragu.

- *ignoreCase*: Postavite na True ako želite da pretraga bude osetljiva na velika i mala slova.

Vraća: Pozicija (indeks) elementa u nizu ako je pronađen, inače 0. Primeri:

```
X = {1, "ABC", 2}
TextWindow.WriteLine({
  Array.IndexOf(X, 10, 1, True),      '0 (Nije pronađeno)
  X.IndexOf(2, 1, True),               '3
  X.IndexOf(1, 2, True),               '0 (Nije pronađeno)
  X.IndexOf("abc", 1, True),           '2
  X.IndexOf("abc", 1, False),          '0 (Nije pronađeno)
  X.IndexOf("ABC", 1, True),           '2
})
```

Array.Indices As Array

Ovo je svojstvo proširenja promenljivih niza koje dobija niz koji sadrži sve indekse za trenutni niz. Imajte na umu da isti posao možete obaviti pozivanjem metode Array.GetAllIndices.

Vraća: Niz popunjen svim indeksima (ključevima) navedenog niza. Vraćeni indeksi niza

počinju od 1.

Primeri: Pretpostavimo da imamo ovaj niz:

```
a = {10, 20, 30, "Adam"}  
a["Ime"] = "Adam"  
a!Godine = 15
```

Njegove indekse možemo dobiti ovako:

```
ids = a.Indices
```

Niz ids će biti

```
{1, 2, 3, 4, "Ime", "Godine"}
```

I možemo dobiti id indekse niza ids samo da bismo bili sigurni:

```
ids2 = Array.GetAllIndices(a.Indices)
```

Niz ids2 će biti

```
{1, 2, 3, 4, 5, 6}
```

Array.IsArray(niz) As Boolean

Dobija da li je data promenljiva niz ili ne.

Parametar:

- *niz*: Promenljiva koju treba proveriti.

Vraća: „True“ ako je navedena promenljiva niz. „False“ u suprotnom. Primeri:

Ova funkcija prima dva parametra `x` i `y` i sabira ih. Ako su x i y numerički ili stringovi, znak + će ih ispravno sabrati, ali ako je jedan od njih niz, znak + će sabrati string reprezentaciju niza, što nije poželjno, pa ćemo u tom slučaju sabiranje izvršiti na drugačiji način. Dakle, potrebna nam je metoda IsArray da bismo znali sa čime imamo posla:

```
Function Add(x, y) ' Dodaje y kao element u x niz, čak i ako je y niz  
If Array.IsArray(x) Then  
Return Array.AddItem(x, y)  
ElseIf Array.IsArray(y) Then  
z[1] = x ' Kreira novi niz i dodaje x u njega,  
Return z.AddItems(y) ' zatim dodaje elemente y niza  
Else  
Return x + y ' Dodaje x i y kao brojeve ili stringove.  
EndIf  
EndFunction
```

Možete isprobati ovu funkciju u aplikaciji `AddArray.sb` u folderu sa primerima.

Array.Join(niz, separator) As string

Spaja date elemente niza u jedan tekst.

Parametri:

- *niz*: Ulazni niz. Ovaj argument se može izostaviti ako pozovete ovu metodu kao metodu proširenja promenljive niza.

- *separator*: String koji se koristi kao separator između elemenata niza. Možete koristiti jedan znak ili više znakova kao separator. Čak možete koristiti i prazan string "" kao separator da biste spojili sve elemente bez ikakvog separatora!

Vraća: String koji sadrži elemente niza, razdvojene datim separatorom. Primer:

```
a = {1, 2, 3}  
x = a.Join(",")  
y = Array.Join({10, 20, 30}, ", ")
```


Nakon izvršavanja gornjeg koda: x će sadržati string: "1, 2, 3", a y će sadržati string: "10, 20, 30".

Array.RemoveItem(niz, index) As Array

Uklanja element niza na navedenom indeksu (ključu).

Parametri:

- *niz*: Ulazni niz. Ovaj argument se može izostaviti ako pozovete ovu metodu kao metodu proširenja promenljive niza.

- *index*: Ključ elementa koji treba ukloniti.

Vraća: Novi niz sa uklonjenim elementom. Originalni niz se neće promeniti. Primer:

```
a = Array.RemoveItem({"a", "b", "c", "d"}, 2)
x = a.Join("")
```

Nakon izvršavanja gornjeg koda, x će sadržati string "acd".

Array.SetItemAt(niz, pozicija, vrednost) As String

Postavlja vrednost elementa niza koji postoji na datoj numeričkoj poziciji. Imajte na umu da će se ulazni niz direktno promeniti, tako da je ovaj metod brži od indeksa niza [] kada se radi sa velikim nizom, ali budite oprezni jer će uticati na referentni niz iz kojeg se kopira trenutni niz! Pogledajte temu o performansama niza za više informacija.

Napomena: ako ova metoda ne radi, verovatno je to zato što ste joj prosledili tekstualnu reprezentaciju niza, a ne stvarni niz.

Ovo je veoma redak slučaj, jer sVB automatski konvertuje tekst u niz. Ali ako ste se nekako susreli sa takvom retkom situacijom, možete je lako popraviti korišćenjem metode Text.ToArray da biste dobili niz iz njegove tekstualne reprezentacije.

Parametri:

- *niz*: Ulazni niz. Ovaj argument se može izostaviti ako pozovete ovu metodu kao metodu proširenja promenljive niza.

- *pozicija*: Broj koji predstavlja poziciju stavke u nizu, koji nije uvek isti kao njen indeks (ključ). Imajte na umu da pozicija mora biti > 0.

- *vrednost*: Vrednost koja se postavlja na stavku.

Vraća: Ključ stavke ako se nalazi na datoj poziciji, u suprotnom prazan string "". Primer:

```
a = {1, 2, 3, "Adam"}
a.SetItemAt(4, "Ali")
```

Nakon izvršavanja gornjeg koda, `a` će sadržati

```
{1, 2, 3, "Ali"}
```

Array.ToString(niz) As String

Konvertuje trenutni niz u string.

Parametar:

- *niz*: Ulazni niz. Ovaj argument se može izostaviti ako pozovete ovu metodu kao metodu proširenja promenljive niza.

Vraća: String reprezentacija niza. Ako je niz jednostavan, njegov string može biti oblika {1, 2, 3}. Ako niz sadrži ključeve, njegov string može biti oblika: {[1]=1, [2]=2, [Ime]=Adam}

Primer:

```
a = {1, 2, 3, "Adam"}  
TextWindow.WriteLine(a.ToStr())  
a["Ime"] = "Adam"  
a!Godine = 15  
TextWindow.WriteLine(a.ToStr())
```

Nakon izvršavanja gornjeg koda, TextWindow će prikazati:

```
{1, 2, 3, Adam}  
{[1]=1, [2]=2, [3]=3, [4]=Adam, [Ime]=Adam, [Godine]=15}
```